USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Failure Handling, and Three Mechanisms That Had Never Fired

UM-NATOS-019

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-019	1.2 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

The kernel had three mechanisms for surviving its own failure: stack guards, a panic handler, and a hang detector. Two of the three had never been observed doing anything, and the third had been silently broken by the second.

This report covers making all three enforce rather than report, and giving each a way to be triggered deliberately. It also records a defect found while building that test harness, which is the most consequential finding here: **the UART receive path was one byte behind, so the interactive shell had never worked from a terminal.**

The unifying failure is not technical. Each of these was verified by observing that it *existed* — a guard word was written, a banner was printed, a watchdog register was armed — rather than by observing it *work*.

Revision 1.2 adds §6 and §7. The panic reason is now written to the persistent record before the handler halts, and drawn on the panel before it spins — so a fault on a board with nothing attached is legible immediately *and* reported again by the next boot, instead of vanishing with the power.

2. Recovery and evidence are in conflict

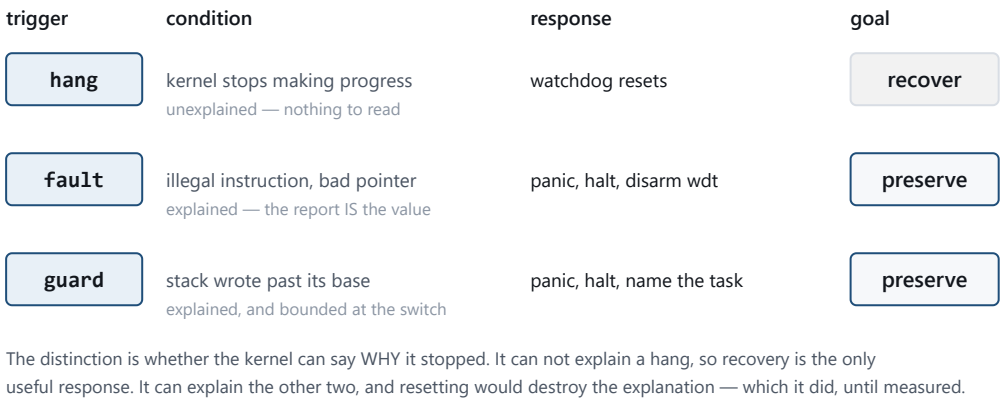

Figure 1. *How the kernel stops, by whether it can explain itself. Recovery and evidence are in direct conflict, and the watchdog silently won until the conflict was measured.*

The hang detector added in UM-NATOS-009 §8 feeds on the scheduler switching between distinct tasks. That is the right signal for a hang. It is also exactly what a **deliberately halted** kernel looks like, and `panic.c` ended with:

```
for (;;) {
    /* Deliberately spin rather than reset: a reset loop would scroll the
     * evidence off the terminal. */
}
```

Two correct decisions, taken months apart, that combine into a wrong one. The comment states the panic handler's entire purpose, and arming the watchdog defeated it without touching that file.

2.1 Measured, not assumed

The obvious move is to disarm the watchdog in the panic path. The less obvious one is to first confirm there is anything to fix — the interaction depends on whether the timer interrupt still fires with `PS.EXCM` set, which is a question about silicon rather than about this code.

With the watchdog left armed and `fault` invoked:

```
*** KERNEL PANIC ***
  exccause : 0  (IllegalInstruction)
  epc      : 0x40083fd0   <- faulting instruction

  halted. reset the board to continue.

rst:0x7 (TG0WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
  [task 0 entered]
```

The report is produced and then destroyed about three seconds later. With `watchdog_disarm()` in the panic path, output stops at `halted.` and stays there through a fourteen-second capture.

This measurement mattered. Had the tick continued to fire during a panic, the fix would have been wrong *and* would have hidden a worse problem — a scheduler switching away from a faulted context and resuming normal operation on it. The result rules that out: no reporter output appears after the panic banner in either configuration, so scheduling genuinely stops.

2.2 The resulting rule

The distinction is whether the kernel can say **why** it stopped:

- it cannot explain a hang, so recovery is the only useful response
- it can explain a fault or a broken guard, and a reset would destroy the explanation

`watchdog_disarm()` exists solely to express that. It is not a general-purpose control, and the header says so.

3. Stack guards: reported, not enforced

`task.c` filled every stack with a pattern and planted a guard word at its base from the beginning. `task_stack_headroom()` was written to walk that pattern. It was never called — both `high_water` figures anywhere in the tree were the *heap's*.

Three separate weaknesses, each individually reasonable:

1. **Coverage.** `kmain` checked guards for `report`, `worker-a` and `worker-b` — three of eight. The display task, which carries the raycaster's call chain, was not among them.
2. **Timing.** The check ran in the reporter, roughly every two seconds. A guard word is broken *after* the damage; a check that runs seconds later reports a corruption whose cause is long gone.
3. **Response.** A break printed `BROKEN` beside the telemetry and execution continued — scheduling tasks whose stacks had already written into a neighbour's, and printing numbers that were by then meaningless.

Checking now happens in `task_schedule()`: every task slot, every switch, and a break calls `kernel_panic_msg()`. A word load per slot per tick is not a cost worth weighing against continuing to run on corrupted memory.

> Written as "all eight tasks" when `TASK_MAX` was 8. It is now 12 (UM-NATOS-021 §5), and the check is written against `TASK_MAX` rather than a literal, so it followed. The prose did not — which is the argument for describing a loop by its bound rather than by the bound's value on the day.

3.1 The margins, which were never known

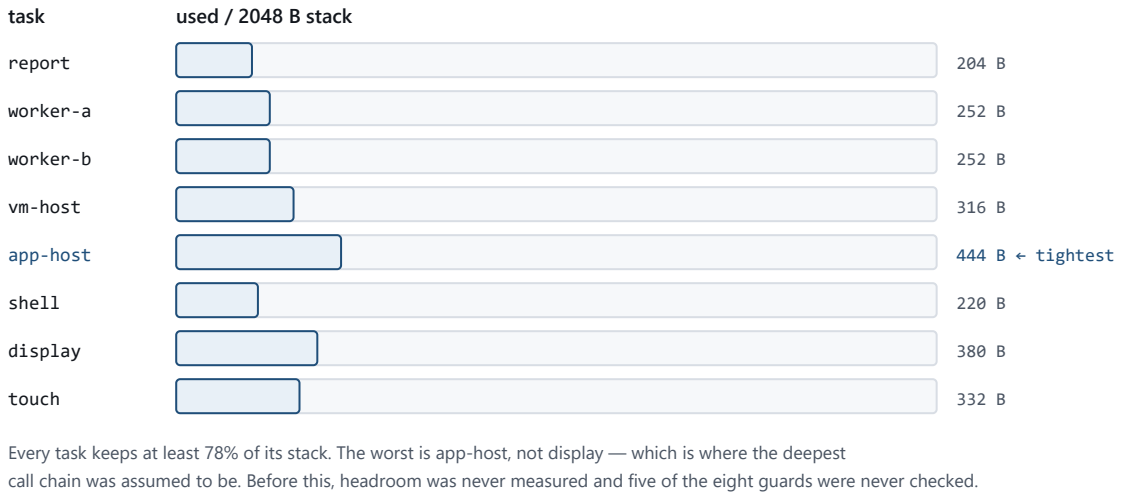

Figure 2. *Measured stack use across all eight tasks. The tightest is not the one that was assumed, which is the argument for measuring all of them.*

```
id  name        free B  of 2048
0   report      1844          4   app-host    1604   <- tightest
1   worker-a    1796          5   shell       1828
2   worker-b    1796          6   display     1668
3   vm-host     1732          7   touch       1716
```

Every task retains at least 78% of its stack; the worst uses 444 B of 2048. The 2 KB figure in `task.h` was a guess, and it turns out to be a comfortable one.

The tightest task is `app-host`, **not** `display` — which is where the deepest call chain was assumed to be when deciding which three tasks were worth checking. That assumption picked the wrong tasks to watch, and is the

argument for measuring all of them rather than the interesting-looking ones.

4. The shell had never worked

While building a harness to send `fault` over the serial line, `mem\r` echoed `mem` and did nothing. `mem\r\n` worked.

That difference is diagnostic. If `\r` triggered execution, then `\r\n` would trigger twice — the second on an empty line, which returns immediately. It working *only* with both bytes means the terminator was not being consumed at all until something followed it.

Confirmed directly:

```
send "mem\r", wait 2.5 s   -> reply BEFORE the extra byte: no
send " ",     wait 2.5 s   -> reply AFTER  the extra byte: YES
```

The receive path ran exactly one byte behind. `uart_getc_nb()` read the RX FIFO through the APB address at `0x3FF40000`; it now reads the AHB alias at `0x60000000`, and `mem\r` replies immediately.

4.1 Why this is the most important finding here

The shell has existed since UM-NATOS-013 §4. In every session since, it has been unusable by a person at a terminal — you press Enter and nothing happens, because **Enter is itself the byte left waiting**. The next keystroke would deliver it, so the symptom is not silence but a one-command lag, which reads as "the board is slow" rather than "the input path is broken".

It survived for two reasons, and both are the same mistake:

- every automated test drove it from a sender emitting **both** CR and LF, which is the one input shape that masks the defect
- the shell was verified by observing that its **banner printed**, not by observing that a command **ran**

A banner proves the task was created and the transmit path works. It says nothing whatsoever about receive. The same substitution — a startup artefact standing in for the behaviour it introduces — is what let the guard checks and the panic handler go unexercised for as long as they did.

5. Triggering failure on purpose

Three shell commands exist solely to make these paths reachable. `hang` predates this work; `fault` and `smash` are new.

Command	Induces	Expected
`hang`	masks interrupts, spins	watchdog resets the board
`fault`	executes `ill`	panic, halt, evidence retained
`smash`	clobbers the running task's guard word	panic at the next switch, naming the task

Verified on hardware:

```
smash  -> *** KERNEL PANIC ***
          reason   : stack guard overwritten
          detail   : 5                      (task 5 is the shell)

fault  -> *** KERNEL PANIC ***
          exccause : 0  (IllegalInstruction)
          halted.                           still halted after 14 s

hang   -> rst:0x7 (TG0WDT_SYS_RESET)        recovered, rebooted

mem\r  -> heap free=71792 ...               no trailing byte needed
```

`smash` correctly identified task 5, which is the shell — the task that invoked it. That is a stronger result than a bare panic, because it shows the scheduler attributing the break to the right task rather than to whichever it happened to be examining.

These are deliberately not compiled out. A destructive command in a development shell is a smaller risk than a recovery path nobody can reach to test.

6. Recording the reason for the next boot

Everything above still assumes a serial cable. A panic prints and halts, which helps only somebody attached at the moment it happens — and this board is normally attached to nothing at all. The panel freezes, the user power-cycles it, and the reason is gone.

The record built in UM-NATOS-018 now carries it:

```
uint32_t fault_kind;    /* exception, guard break, or none    */
uint32_t fault_detail;  /* exccause, or the task id           */
uint32_t fault_epc;     /* faulting instruction               */
uint32_t fault_boot;    /* which boot it happened on          */
```

6.1 Recorded before it is reported

Both panic entry points write the record **before** printing anything. The ordering is deliberate: a handler that reports first and records second loses the record if it dies while reporting, and that is not a remote possibility. The UART path is the more complicated of the two, and the system reaching this code is by definition in a state nobody designed.

The reverse ordering would also be self-concealing — a panic that died during its own report would leave no record AND no output, which is indistinguishable from a fault that never happened.

6.2 Not cleared by being read

A fault stays on the record until a different one replaces it. Clearing it after one report would mean that power-cycling past the message destroys it, and the user who most needs this feature is exactly the one who reboots twice before thinking to attach a cable.

`fault_boot` is what keeps that honest. Without it, a fault from thirty boots ago reads identically to one from the last run.

6.3 Verified

```
smash        -> recorded : yes
next boot    -> LAST FAULT : stack guard overwritten, task 5  (boot #6)

fault        -> epc 0x40084071
next boot    -> LAST FAULT : exception, exccause 0, epc 0x40084071  (boot #8)
boot again   -> same line, unchanged
```

Three properties, each with a distinct failure mode: the guard break survived a reset; the exception **replaced** it rather than being ignored or appended; and the second reboot proves reading does not consume the record. The reported `epc` matches what the panic itself printed, which is what rules out the record being written from stale or default state.

Bumping the record from version 1 to 2 incidentally exercised the version check that UM-NATOS-018 §2 describes but had never triggered: the old record was rejected and reset cleanly rather than being read with the wrong field layout.

7. Putting it on the panel

§6 makes a fault legible on the *next* boot. It does nothing for the person holding a frozen device now, and "the kernel panicked" and "the renderer stopped" look identical from across the room.

The panic handler now draws to the display. That is a larger claim than it sounds, because the display driver is built for a healthy system and a panic is by definition not one.

7.1 Three assumptions suspended at once

`display_enter_panic_mode()` turns off, permanently:

Assumption	Why it cannot hold
the draw lock	Blocking on a mutex means `task_block()` and a yield, and the scheduler has stopped. Waiting on a lock whose owner will never run again is a hang — and a hang here costs the report.
DMA	The descriptors may describe a transfer already in flight, or a buffer belonging to whatever just died.
the unbounded FIFO wait	`spi2_tx()` spins until the controller retires a transfer. Correct while the system is healthy; an infinite loop once it is not.

The switch is one-way. There is no `display_leave_panic_mode()`, because a driver that can be talked back out of panic mode invites somebody to try, and the kernel is on its way to a halt in any case.

`display_ready()` guards the whole thing: a panic before `display_init()` has completed draws nothing rather than writing to an unconfigured controller.

7.2 Ordered by decreasing reliability

The handler does three things, in this order:

1. **write the flash record** — fewest moving parts, and the only one that survives the power cycle

2. **print to the UART** — one peripheral, no memory beyond a string literal already resident in DRAM
3. **draw the panel** — needs the SPI controller, the flash-mapped font, and a peripheral whose state nobody has verified

Each step can only cost the steps after it. If drawing wedges despite the bounds in §7.1, the record and the serial report have both already happened. The reverse order would put the most fragile step first and risk everything on it.

7.3 Measured, because a screen cannot be queried

Nobody can ask a halted board what it drew, and by eye "the layout is wrong" and "the driver never ran" are the same observation. So the handler reports the byte count:

```
fault -> panel : 175955 bytes drawn
smash -> panel : 176704 bytes drawn
```

A full repaint is 320 × 240 × 2 = 153,600 bytes; the remainder is the title bar and the text. A number near zero would mean the panic never reached the panel at all. This converts an unfalsifiable visual impression into a claim that fails loudly — the same move as the boot counter in UM-NATOS-018 §2.2.

Screen content confirmed visually on hardware: title bar, the cause name, `cause` and `pc` in hex, and a note that the reason is saved for the next boot.

7.4 A hang in the code that prevents hangs

Redirecting the driver's internal `mutex_lock(&g_lock)` calls to the new panic-aware `draw_lock()` was done with a regular expression, which also rewrote the body of `draw_lock` itself:

```
static void draw_lock(void)
{
    if (!g_panic_mode) {
        draw_lock();          /* <- was mutex_lock(&g_lock) */
    }
}
```

Infinite recursion, inside the function whose entire purpose is to avoid a hang. The board wedged in `display_init()` before printing a single self-test.

It was found by bisecting — reverting `display.c` alone with `git checkout` and rebuilding — rather than by reading the diff, which is the faster route once a symptom is "nothing happens at all". It is also the fourth scripted edit in this project to fail silently, and the reason those edits now carry assertions that stop on a missed match rather than writing an unchanged file.

8. Metrics

Quantity	Value
Tasks with guards checked, before → after	3 → 8
Guard check frequency, before → after	~2 s (reporter) → every context switch
Guard check cost	8 word loads per tick
Stack headroom, worst case	1,604 B free of 2,048 (`app-host`)
Stack headroom, best case	1,844 B free of 2,048 (`report`)
Minimum margin across all tasks	78%
UART rx lag, before → after	1 byte → 0
Sessions the shell was unusable interactively	every one since UM-NATOS-013
Failure paths now triggerable on demand	3
Fault fields added to the persistent record	4
Record version	1 → 2
Panic paths that record before reporting	2 of 2
Driver assumptions suspended in panic mode	3
Bytes drawn by a panic screen	~176,000 (153,600 is one full repaint)
Reporting steps, ordered by reliability	3

9. What this does not establish

- **No pre-emptive overflow detection.** A guard word is found *after* it has been overwritten. The margins in §3.1 say that is not close to happening, but the mechanism is still a post-mortem, not a barrier. A guard page would be a barrier; this hardware has no MMU to provide one.
- **No wild-pointer protection.** `task.h` has always said this: a native task can corrupt any other. Guards catch the common case — growth past a stack's own base — and nothing catches a stray write into the middle of a neighbour.
- **The panic screen has no failure path of its own.** If the panel is wedged, §7.3's byte count is what says so — but only to somebody with a serial cable, which is the audience the screen exists to replace.
- **Panic mode is untested against a genuinely broken controller.** §7.1 removes the ways drawing could hang in theory. Every test so far ran on a display that was working perfectly at the moment of the fault, so the bounds have never actually been reached.
- **A fault during the recording itself is not survivable.** §6.1 orders the write before the report, which covers a panic that dies while printing. It does not cover one that dies inside `store_record_fault()` — the checksum would reject the torn record on the next boot, correctly, and the reason would be lost.
- **Only two fault kinds are recorded.** A hang recovered by the watchdog writes nothing at all, so the most common recoverable failure is the one the record cannot describe.

- **No stack sizing per task.** All eight get 2 KB regardless of measured use. The figures in §3.1 would support trimming several, but nothing depends on reclaiming that memory yet.
- **The AHB alias is validated behaviourally, not from documentation.** The measurement establishes that the APB address lags and the AHB alias does not. It does not establish *why*, and no erratum has been cited here to support it.

10. References

- UM-NATOS-009 §8 — the hang detector this reconciles with the panic path
- UM-NATOS-013 §4 — the shell and its polled UART receive, verified by its banner
- UM-NATOS-018 §5 — the previous instance of a harness lying about its own results
- UM-NATOS-017 §6 — the standing rule on instruments reporting themselves invalid
- `kernel/task.c` — `task_stack_broken()`, `task_stack_tightest()`, checked in `task_schedule()`
- `kernel/panic.c` — `halt_forever()`, `kernel_panic_msg()`
- `kernel/uart.c` — `UART_FIFO_AHB_REG` and the reasoning above it
- UM-NATOS-018 §2 — the record this extends, and its version check
- `kernel/store.c` — `store_record_fault()`, written from the panic path
- `kernel/display.c` — `display_enter_panic_mode()`, `draw_lock()`, the bounded FIFO wait
- `kernel/panic.c` — `panic_screen()` and the ordering in `halt_forever()`